

PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA INFORMÁTICA

GameLluvia: Guardián de la Lluvia

Informe Técnico de Arquitectura, Patrones de Diseño y Modelado de Software

Integrantes:

Rodrigo Escudero

Ariel Fuenzalida

Matías Santelices

Octubre de 2026

Índice

Lista de figuras **2**

Lista de tablas **3**

Introducción **4**

1 GM-1: ¿Qué experimenta el jugador? **5**

2 GM-2: ¿Cómo se transformó el código base? **10**

3 GM-3: Diagramas de clases y arquitectura del sistema **13**

4 GM-4: ¿Qué abstracciones definen el comportamiento? **17**

5 GM-5: ¿Cómo se protegen los datos y se organiza el código? **19**

6 GM-6: ¿Por qué centralizar la configuración y el audio? (Singleton) . **21**

7 GM-7: ¿Cómo se desacoplan los controles y comportamientos? (Strategy) **24**

8 GM-8: ¿Cómo se define el esqueleto de la caída y captura? (Template Method) **26**

9 GM-9: ¿Cómo se construyen niveles complejos de forma legible? (Builder) **28**

10 Control de Versiones **30**

Índice de figuras

Figura 1.1 Evolución visual de los biomas climáticos y la batalla final en el espacio 6
Figura 1.2 Modalidades adicionales: Tienda de desbloqueos y Versus local para dos jugadores 9
Figura 3.1 Mapa General de Arquitectura: Distribución de subsistemas y dependencias principales en el módulo core 14
Figura 3.2 Vista detallada 1: Clases del modelo de dominio, jerarquía de gotas, reglas de partida y estrategias de control 15
Figura 3.3 Vista detallada 2: Subsistema de combate espacial contra el Jefe OVNI, componentes de vista y persistencia del perfil 16
Figura 6.1 Diagrama UML del patrón Singleton aplicado a la configuración global y audio 23
Figura 7.1 Diagrama UML del patrón Strategy aplicado a los controles dinámicos, física de terrenos y fases del jefe 25
Figura 8.1 Diagrama UML del patrón Template Method multietapa en la jerarquía Gota 27
Figura 9.1 Diagrama UML del patrón Builder con Director en el enum Dificultad 29
Figura 10.1 Trazabilidad e historial de commits en GitHub 30

Índice de tablas

Tabla 1.1 Catálogo de elementos, efectos y variación del estado del jugador 7
Tabla 2.1 Comparativa técnica entre el prototipo base y la arquitectura desarrollada (GM) 11

Introducción

El presente informe técnico expone la arquitectura, modelado conceptual, patrones de diseño e implementación del videojuego **GameLluvia: Guardián de la Lluvia**, un software interactivo desarrollado en el lenguaje de programación Java utilizando el framework multimedia **LibGDX**.

A partir de un prototipo base simple —el cual presentaba limitaciones de acoplamiento, fallos en la eliminación de elementos e iteraciones en colecciones, y ausencia de estados de juego estructurados— aplicamos una reingeniería profunda del software. Esta transformación dio lugar a un sistema modular, robusto y testeable, que incorpora una campaña procedural de 30 niveles distribuida en tres biomas (Llanura, Hielo y Desierto de Atacama), una batalla climática final contra un Jefe OVNI en el espacio exterior, un modo competitivo Versus local en pantalla dividida y un desafío oculto de alta dificultad.

El propósito de este documento es proporcionar una radiografía exhaustiva de ingeniería de software que explique técnica y conceptualmente cómo y por qué fue diseñado cada componente del sistema:

- **Experiencia y Reglas de Dominio:** Análisis detallado de las mecánicas de juego, controles en múltiples plataformas, catálogo de elementos, penalizaciones físicas por deshidratación y modos especiales.
- **Evolución y Refactorización del Código:** Comparativa técnica de la transición desde una estructura monolítica hacia una arquitectura en capas desacopladas.
- **Modelado Estático (UML):** Diagramación formal de las clases del dominio, explicitando el uso de las abstracciones nativas de LibGDX y la separación entre lógica y renderizado.
- **Abstracciones Polimórficas:** Justificación e integración de la clase abstracta (`Gota`) y de la interfaz (`ControlMovimiento`), demostrando su rol activo en tiempo de ejecución.
- **Buenas Prácticas, GRASP y SOLID:** Análisis de encapsulamiento estricto, cohesión, bajo acoplamiento y cumplimiento de los cinco principios de diseño orientado a objetos.
- **Patrones de Diseño GoF:** Implementación justificada de cuatro patrones de diseño (**Singleton**, **Strategy**, **Template Method** y **Builder**), complementados con el contraste frente a sus respectivos antipatrones.
- **Control de Versiones y Trazabilidad:** Gestión del ciclo de desarrollo y repositorio del proyecto en Git y GitHub.

1. GM-1: ¿Qué experimenta el jugador?

Narrativa y ambientación: La odisea del Guardián y Luma

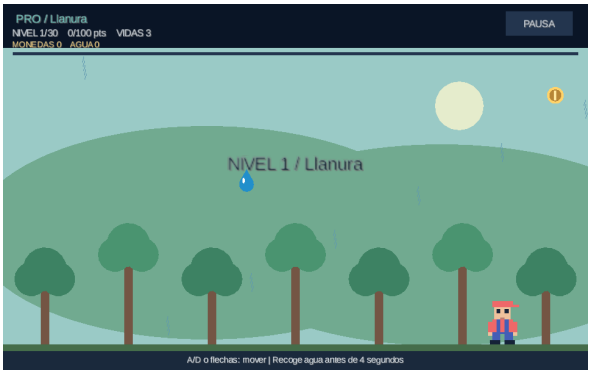
El videojuego **GameLluvia: Guardián de la Lluvia** sumerge al jugador en una atmósfera de ciencia ficción postapocalíptica. Un planeta antaño fértil ha sido víctima de una invasión extraterrestre: una gigantesca nave nodriza alienígena (el **Jefe OVNI**) se ha posicionado en la estratosfera para drenar las reservas de agua potable y corromper el ciclo hidrológico de la atmósfera terrestre mediante precipitaciones tóxicas y lluvia ácida.

El usuario encarna al **Guardián de la Lluvia**, un recolector de élite que emprende una travesía a lo largo del globo terráqueo guiado por **Luma**, la guardiana elemental del agua (representada visualmente en el juego con su túnica ceremonial turquesa, capucha y emblema sagrado de gota). La misión consiste en capturar la mayor cantidad de agua potable pura, esquivar sustancias letales, subsistir a las condiciones climáticas de cada bioma y acumular la energía hídrica suficiente para ascender a la órbita superior y destruir a la nave invasora.

Progresión geográfica: Tres biomas y el ascenso espacial

La campaña principal se estructura en 30 niveles procedurales distribuidos en tres regiones ecológicas, culminando en la estratosfera:

1. **Bioma Llanura (Niveles 1 a 10):** Inicio de la travesía sobre praderas templadas. El jugador aprende los fundamentos de recolección y esquiva, enfrentando lluvia ácida básica, sangre contaminante y gotas venenosas, mientras equipa su primera arma de defensa: el **Pulso de Llanura**.
2. **Bioma Hielo (Niveles 11 a 20):** Cordillera glacial azotada por ventiscas polares. Las gotas de nieve ralentizan el paso y los fragmentos de hielo inmovilizan por completo al jugador. Para sobrevivir es imperativo recoger termos de aislamiento térmico y empuñar la **Espada de Hielo**.
3. **Bioma Desierto de Atacama (Niveles 21 a 30):** Clima árido extremo con sol abrasador y vientos laterales constantes. Caen proyectiles de fuego y espinas de cactus que infligen daño directo al impacto. El personaje debe buscar oasis para frenar la deshidratación y utiliza la **Pistola Láser Solar**.
4. **Ascenso y Batalla Final en el Espacio (Nivel 31):** Al superar los 3.000 puntos (100 puntos acumulados por cada nivel), el Guardián activa su propulsión hacia la alta atmósfera. Allí enfrenta cara a cara a la nave nodriza extraterrestre, armado con el devastador **Cañón Hidro-Plasma**.



(a) Campaña en Bioma Llanura



(b) Campaña en Bioma Hielo



(c) Campaña en Desierto de Atacama



(d) Combate espacial contra el Jefe OVNI

Figura 1.1: Evolución visual de los biomas climáticos y la batalla final en el espacio

Controles de entrada y multiplataforma

El jugador controla al personaje horizontalmente en la base del escenario mediante dos modalidades complementarias:

- **Modo Teclado (Escritorio):** Desplazamiento con teclas `A/D` o flechas de dirección `LEFT/RIGHT`. En combate espacial, se dispara proyectiles de agua a presión con la `Barra Espaciadora` o clic izquierdo del ratón.
- **Modo Táctil / Ratón:** El personaje sigue suavemente el dedo o cursor proyectado en las coordenadas virtuales del mundo. Al pulsar sobre la zona superior se efectúan los disparos.
- **Alternancia en Caliente:** En cualquier instante de la partida, el usuario puede pulsar la tecla `T` o el botón del HUD para alternar inmediatamente entre control por teclado y control táctil.
- **Pausa y Sonido:** La partida se pausa o reanuda con `P` o `ESC`. La música y efectos se silencian globalmente con `M`.
- **Modo Versus Local (2 Jugadores):** Pantalla dividida en el mismo monitor: el Jugador 1 se mueve con `A/D` y dispara con `W`; el Jugador 2 se mueve con flechas y

dispara con flecha arriba (`UP`).

Catálogo de elementos atmosféricos y efectos

Cada elemento que se precipita desde el firmamento cuenta con propiedades visuales y mecánicas diferenciadas:

Tabla 1.1: Catálogo de elementos, efectos y variación del estado del jugador

Elemento	Efecto al capturarlo	Variación
Agua Pura (Gota-Buena)	Suma puntos, recarga tanque de agua y sacia la sed.	+10 pts, +1 agua
Agua Doble	Duplica la recarga hídrica y otorga bonificación de puntaje.	+20 pts, +2 agua
Oasis	Hidratación profunda y gran bonificación en zonas áridas.	+30 pts, +3 agua
Moneda Dorada	Moneda acumulable para desbloquear aspectos en la tienda.	+1 o +2 monedas
Café / Fruta	Acelera la velocidad del personaje en 1,5× durante 5 s.	Velocidad +50 %
Termo	Confiere inmunidad térmica contra el frío durante 6 s.	Inmunidad hielo
Sangre	Contaminante biológico que drena puntaje acumulado.	−20 puntos
Veneno	Intoxicación química: reduce la velocidad del personaje a la mitad.	−10 pts, vel. −50 %
Nieve	Enfriamiento polar: entorpece la movilidad del personaje.	−20 pts, vel. −50 %
Hielo	Congelamiento severo: inmoviliza al jugador totalmente.	−30 pts, vel. = 0
Ácido	Químico corrosivo: destruye una vida e inflige daño.	−1 vida, −30 pts
Fuego / Cactus	Amenazas térmicas y desérticas de impacto directo.	−1 vida, −40/ − 25 pts

Dinámica de dificultad y penalización por sed

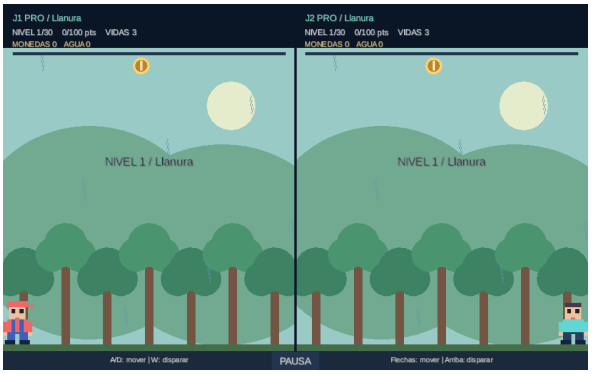
1. **Mecánica de Sed y Deshidratación:** Si el personaje pasa más de 4 segundos sin recolectar agua pura (`sinAgua >4`), entra en fatiga por sed. Su velocidad decae progresivamente hasta un 60 %, obligando a una búsqueda activa incesante. Si supera los 12 segundos sin beber, comienza a recibir daño periódico por deshidratación cada 4 segundos.
2. **Ronda de Bonificación Dorada (Bonus Secreto):** Cada 5 niveles superados (niveles 5, 10, 15, 20 y 25), la lluvia tóxica se detiene y se activa una tormenta dorada de 8 segundos donde solo caen monedas para la tienda.
3. **Tres Niveles de Dificultad Inicial:**
   - *Noob:* 5 vidas iniciales, caída moderada (200 px/s) y 16 % de probabilidad de peligros.
   - *Pro:* 3 vidas iniciales, velocidad estándar (290 px/s) y 25 % de peligros.
   - *Hacker:* 1 sola vida (un golpe causa derrota instantánea), caída veloz (370 px/s) y 34 % de peligros.

Desafíos especiales y personalización de personajes (Skins)

- **Combate contra el Jefe OVNI:** Al alcanzar los 3.000 puntos, la nave nodriza ataca con ráfagas guiadas. Al descender al 65 % de vida entra en furia acelerando su trayectoria, y al 35 % despliega dos drones subordinados que activan un escudo impenetrable mientras regeneran la nave nodriza. El jugador debe derribar los drones antes de rematar al jefe.
- **Desafío Oculto del Alien:** Un micro-botón secreto oculto en la interfaz del menú principal permite enfrentarse de inmediato al Jefe OVNI en dificultad Pro con 40 unidades de munición iniciales. No concede dinero ni récord de campaña: su único galardón es desbloquear el aspecto legendario `Alien`.
- **Tienda y Galería de Héroes (30 Aspectos):** Catálogo de personalización estética que incluye figuras icónicas como Explorador, Luma, Mario, Sonic, Steve, Spiderman, Iron Man, Goku, Gojo y Alien, desbloqueables mediante monedas o alcanzando hitos en cada dificultad.



(a) Tienda de aspectos y personalización de sprites



(b) Modo competitivo Versus en pantalla dividida

Figura 1.2: Modalidades adicionales: Tienda de desbloqueos y Versus local para dos jugadores

2. GM-2: ¿Cómo se transformó el código base?

Diagnóstico del prototipo base y deficiencias arquitectónicas

El proyecto se construyó a partir del prototipo elemental de lluvia propuesto en el tutorial oficial de LibGDX. Si bien dicho código constituía una demostración funcional mínima, presentaba severas deficiencias arquitectónicas que impedían su escalamiento hacia un videojuego completo y mantenible:

1. **Monolito y Acoplamiento Extremo (Espagueti):** Una única clase principal concentraba simultáneamente la carga de texturas y sonidos, la lectura de eventos de teclado, la física de colisiones, el cálculo de puntajes y el bucle de renderizado. Modificar el dibujo de una gota conllevaba el riesgo directo de quebrar las rutinas de colisión o el bucle de audio.
2. **Estructuras Primitivas y Listas Paralelas:** Las posiciones de las gotas y sus categorías se administraban en colecciones paralelas desacopladas (`Array<Rectangle>` para geometría y arreglos de tipos primitivos para estados). Esta dispersión propiciaba desincronizaciones de estado ante eliminaciones o desbordamientos.
3. **Anomalía de Eliminación con Salto de Índices (*Index Shifting*):** El bucle de simulación iteraba de forma ascendente (`for (int i = 0; i <gotas.size; i++)`). Al remover un elemento impactado en la posición i, los elementos posteriores se desplazaban una posición hacia la izquierda, provocando que la gota contigua $(i + 1)$ fuera omitida en ese fotograma, cayendo sin evaluar colisión.
4. **Física Dependiente de la Tasa de Cuadros (FPS):** Los temporizadores de invulnerabilidad se calculaban decrementando un contador de fotogramas fijo (`invulnerabilidad = 60`). En pantallas de 120 Hz el tiempo de protección duraba la mitad, mientras que ante caídas de rendimiento el jugador quedaba desprotegido más tiempo del estipulado.
5. **Ausencia de Ciclo de Vida y Máquina de Estados:** No existía separación entre menú, selección de dificultad, pausa, victoria o derrota; la partida comenzaba de forma abrupta e inmediata al abrir la ventana.

Decisiones de rediseño y justificación técnica

Para resolver estas limitaciones, se implementó una reingeniería exhaustiva fundamentada en patrones de diseño GoF y principios SOLID:

Tabla 2.1: Comparativa técnica entre el prototipo base y la arquitectura desarrollada (GM)

Dimensión	Prototipo Base (LibGDX)	Solución Implementada (GM)
Gestión de partida y estado	Variables sueltas en la clase principal; paso manual de parámetros dispersos.	Clase `Partida` encapsula reglas de sesión. `GameLluvia` administra máquina de 10 estados (`FSM`).
Ciclo de vida de las gotas	Bloque condicional extenso (`if-else`) evaluando tipos manualmente.	Patrón **Template Method**: clase abstracta `Gota` fija la plantilla de actualización en 4 pasos primitivos.
Eliminación en colecciones	Recorrido ascendente con eliminación que saltaba elementos contiguos.	**Recorrido inverso seguro** (`for (int i = size - 1; i >= 0; i-)`) en `Lluvia.java`, eliminando el salto de índices sin instanciar iteradores.
Control de movimiento	Lectura cableada de teclas acoplada directamente al modelo del personaje.	Patrón **Strategy**: interfaz `ControlMovimiento` con implementaciones desacopladas e intercambiables en caliente (`T`).
Construcción de niveles	Constantes numéricas fijas dispersas en el código de simulación.	Patrón **Builder**: `Nivel.Builder` valida invariantes; `Dificultad` actúa como Director de 30 niveles.
Configuración y persistencia	Lecturas descontroladas de disco; riesgo de desincronización de audio.	Patrón **Singleton**: `ConfiguracionJuego` centraliza preferencias atómicas y sincronización de música.
Separación Vista / Lógica	Mezcla total de OpenGL con física y reglas de colisión.	**Separación estricta**: capa de modelo 100 % agnóstica de gráficos, permitiendo ejecución de 36 pruebas unitarias sin GPU.

Organización del sistema en cuatro capas de responsabilidad

1. **Capa de Modelo (Dominio):** Clases `Partida`, `Personaje`, `Lluvia`, `Gota`, `Nivel`, `JefeOvni` y `ChorroAgua`. Operan exclusivamente sobre física matemática, cajas

delimitadoras (`Rectangle`) y reglas lúdicas. No importan controladores gráficos ni subsistemas de sonido.

2. **Capa de Vista (Presentación Gráfica):** Centralizada en `VistaJuego`, auxiliada por `DibujoPersonaje`, `DibujoEscenario` y `DibujoObjetos`. Interpreta el estado numérico del modelo y renderiza formas vectoriales con `ShapeRenderer` y textos con `SpriteBatch`.
3. **Capa de Control (Orquestación y Entrada):** `GameLluvia` administra la máquina de estados (`MENU`, `DIFICULTAD`, `TIENDA`, `PUNTUACIONES`, `AYUDA`, `JUGANDO`, `PAUSA`, `DERROTA`, `VICTORIA`, `RESULTADO_VERSUS`) y conecta los periféricos físicos con las estrategias de control.
4. **Capa de Persistencia:** `ConfiguracionJuego` y `PerfilJugador` interactúan con la interfaz `Preferences` de LibGDX para serializar récords, monedas acumuladas y aspectos desbloqueados de forma atómica.

Beneficios concretos de ingeniería de software

- **Alta Mantenibilidad y Bajo Acoplamiento:** Modificar los esquemas estéticos de los personajes en `DibujoPersonaje` no altera la lógica de daño del `Personaje`.
- **Extensibilidad Abierta/Cerrada (OCP):** Incorporar una nueva amenaza o mejora solo requiere crear una subclase de `Gota` o una entrada en `TipoObjeto`, sin tocar el bucle de físicas de `Lluvia`.
- **Eficiencia en Tiempo Real (Cero Garbage Collection):** El recorrido inverso en colecciones nativas evita la instanciación de objetos `Iterator` por cuadro, suprimiendo pausas por recolección de basura a 60 FPS.
- **Testeabilidad Total (36 Pruebas Automatizadas):** Al desacoplar completamente el dominio del contexto gráfico de LibGDX, todas las reglas de negocio, progresión de niveles y penalizaciones por sed se verifican automáticamente en la suite de pruebas unitarias.

3. GM-3: Diagramas de clases y arquitectura del sistema

Abstracciones de infraestructura utilizadas de LibGDX

El proyecto se apoya en los módulos nativos provistos por el motor LibGDX para la gestión del ciclo de vida, renderizado acelerado por hardware y cálculos geométricos:

- **`ApplicationAdapter`:** Clase base del ciclo de vida del juego (`create`, `render`, `resize`, `dispose`).
- **`FitViewport`:** Administra una resolución virtual de 960×600 píxeles, garantizando una relación de aspecto 16:10 invariante mediante barras negras de compensación (*letterboxing*) en cualquier monitor.
- **`SpriteBatch` y `ShapeRenderer`:** Motores gráficos de bajo nivel encargados del renderizado de tipografías (`BitmapFont`) y geometrías vectoriales sólidas respectivamente.
- **`OrthographicCamera`:** Provee la matriz de proyección bidimensional ortogonal para el renderizado del mundo y la interfaz gráfica.
- **`Rectangle` y `Vector2`:** Estructuras matemáticas optimizadas para el cálculo de cajas delimitadoras ($AABB$) y detección de intersecciones en colisiones.
- **`Preferences`:** Subsistema de almacenamiento persistente clave-valor para guardar récords, monedas acumuladas y aspectos cosméticos.
- **`Sound` y `Music`:** Controladores para la reproducción sincronizada de efectos sonoros puntuales y pistas musicales continuas en bucle.

Inventario exhaustivo de las 29 clases del módulo core

El sistema implementa íntegramente 29 clases, enumeradores e interfaces en el paquete `puppy.code`:

1. **Flujo y Orquestación:** `GameLluvia` (controlador de ciclo de vida) y `GameLluvia.Estado` (máquina de 10 estados).
2. **Capa de Dominio y Simulación:** `Partida` (reglas de sesión individual), `Partida.Fase` (fases de campaña), `PartidaVersus` (competitivo local), `Personaje` (entidad del jugador), `Hidratacion` (estados de sed), `Lluvia` (gestor de colecciones), `Gota` (clase base abstracta), `GotaBuena`, `GotaMala`, `ObjetoEspecial`, `TipoObjeto` (catálogo de ítems y amenazas), `Nivel`, `Nivel.Builder` (construcción fluida de niveles), `Dificultad` (modos de juego) y `Zona` (biomas climáticos).
3. **Subsistema de Controles y Terreno:** `ControlMovimiento` (interfaz de estrategia), `ControlTeclado`, `ControlTactil` y `MovimientoTerreno` (estrategia de física según el bioma).

4. **Subsistema de Combate Espacial:** `JefeOvni` (entidad del jefe), `JefeOvni.Fase` (estrategia de fases del jefe), `SubordinadoOvni` (drones de escolta con escudo), `ChorroAgua` (proyectiles del jugador) y `ArmaBioma` (estrategia de armas elementales).
5. **Capa de Presentación Gráfica:** `VistaJuego` (fachada de renderizado), `DibujoPersonaje`, `DibujoEscenario` y `DibujoObjetos`.
6. **Capa de Persistencia y Perfil:** `ConfiguracionJuego` (Singleton centralizado), `PerfilJugador` y `Aspecto` (catálogo de 30 skins).

Mapa General de Arquitectura (Distribución de Subsistemas)

La figura 3.1 presenta la vista arquitectónica global del sistema, explicitando la interacción de alto nivel entre los seis subsistemas del juego:

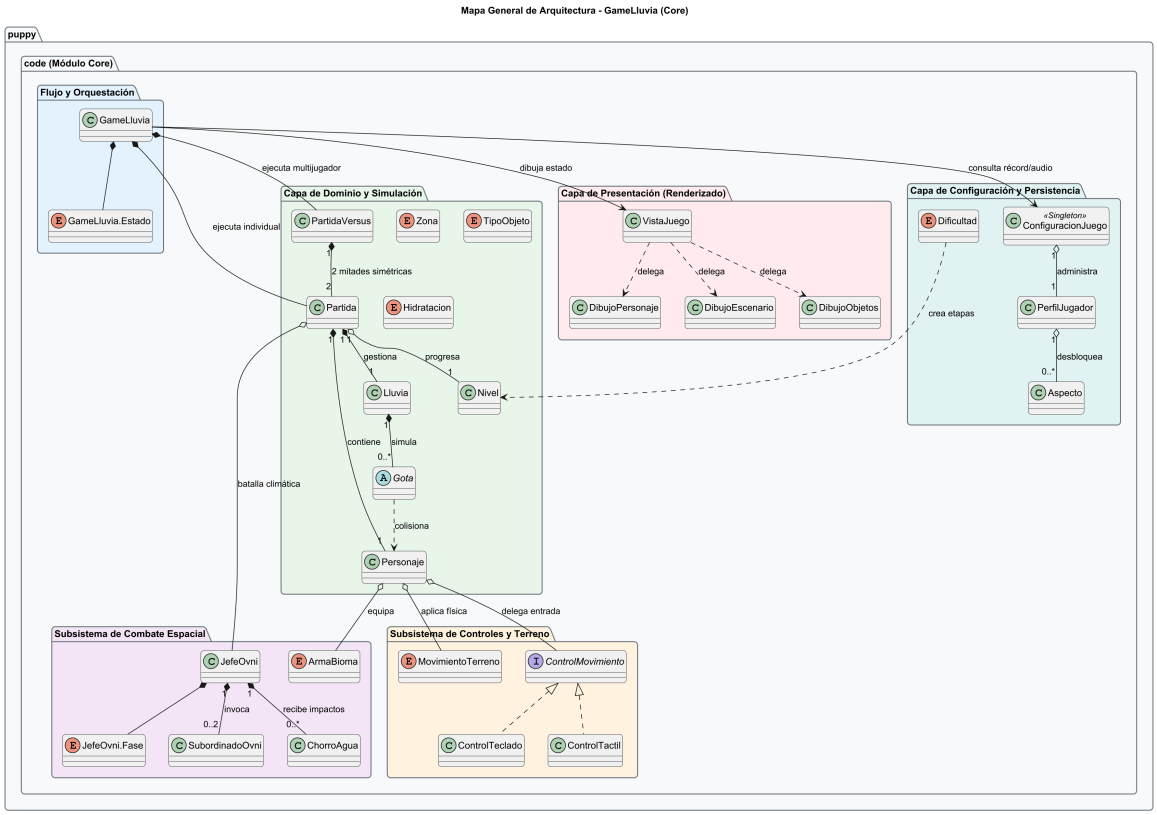


Figura 3.1: Mapa General de Arquitectura: Distribución de subsistemas y dependencias principales en el módulo core

Vista 1: Dominio, Reglas de Juego, Lluvia y Controles

La figura 3.2 detalla el modelado estático de la simulación, la gestión polimórfica de la lluvia y el desacoplamiento de la entrada del usuario:

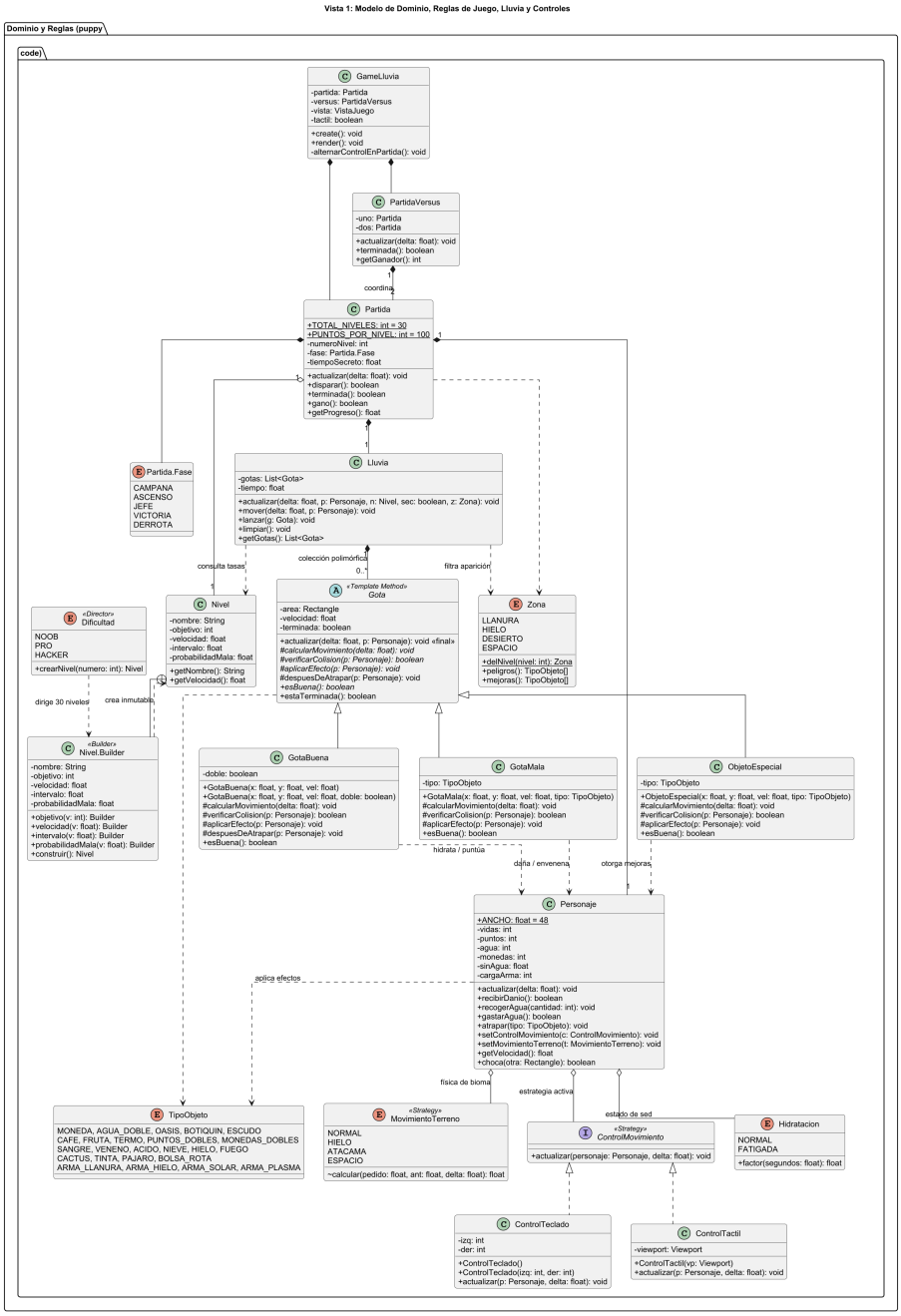


Figura 3.2: Vista detallada 1: Clases del modelo de dominio, jerarquía de gotas, reglas de partida y estrategias de control

Vista 2: Combate Espacial, Presentación Gráfica y Persistencia

La figura 3.3 modela la batalla final contra la nave nodriza alienígena, los módulos de renderizado vectorial y la capa de almacenamiento atómico:

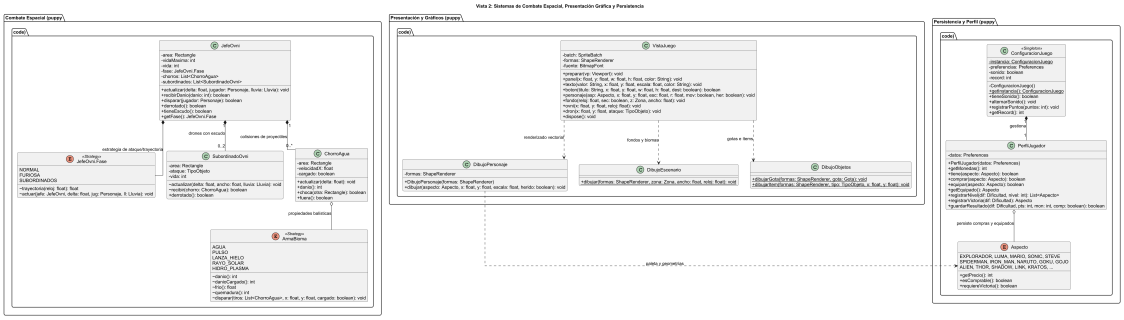


Figura 3.3: Vista detallada 2: Subsistema de combate espacial contra el Jefe OVNI, componentes de vista y persistencia del perfil

4. GM-4: ¿Qué abstracciones definen el comportamiento?

¿Por qué una clase abstracta para las gotas?

En el dominio del videojuego nunca cae una "gota genérica" sin propósito. Cada elemento que desciende de la atmósfera debe pertenecer a una variante concreta (agua pura, objeto especial con efectos, monedas o contaminantes) que determine su consecuencia al impactar al jugador.

Por esta razón se diseñó la clase `Gota` como abstracta. Ella encapsula los atributos y métodos comunes a todas las variantes, pero no puede ser instanciada directamente.

La clase abstracta define:

- **Atributos privados:** Delimitador de colisión (`Rectangle area`), tasa de caída vertical (`float velocidad`) y estado de ciclo de vida (`boolean terminada`).
- **Métodos concretos de consulta y física base:** `estaTerminada()`, `getX()`, `getY()`, junto a rutinas auxiliares finales como `caer(delta, factor)` y `toca(personaje, margen)`.
- **El método plantilla `actualizar()`:** Declarado como `final`, fija la secuencia algorítmica invariable de caída, escape inferior y evaluación de colisiones.
- **Operaciones primitivas abstractas:** `calcularMovimiento(delta)`, `verificarColision(personaje)` y `aplicarEfecto(personaje)`, delegadas obligatoriamente en las clases hijas, complementadas con el método gancho opcional (*hook*) `despuesDeAtrapar(personaje)`.

De esta forma, cualquier subclase hereda automáticamente la estructura algorítmica, debiendo únicamente ocuparse de programar sus pasos primitivos particulares.

¿Qué papel cumple la interfaz `ControlMovimiento`?

El desplazamiento del personaje depende del entorno en el que se ejecute el juego: en computadores se emplean teclas físicas, en dispositivos móviles se realiza arrastre táctil y en partidas multijugador dos usuarios comparten el mismo teclado con mapeos diferentes.

Para evitar que la clase `Personaje` se llene de condicionales dependientes del hardware, se definió la interfaz `ControlMovimiento`. Ella declara un único método: `actualizar(Personaje personaje, float delta)`.

¿Cuántas clases la implementan?

Actualmente existen dos implementaciones activas que operan en tiempo de ejecución:

1. **`ControlTeclado`:** Lee el estado de las teclas en `Gdx.input`. Posee sobrecarga de constructores: una variante por defecto que responde a `A/D` y flechas, y una variante parametrizada (`int izq, int der`) utilizada para instanciar a los dos contrincantes del modo Versus en un mismo teclado.
2. **`ControlTactil`:** Rastrea los toques en pantalla proyectando las coordenadas virtuales mediante la cámara (`viewport.unproject()`), desplazando suavemente al personaje hacia el dedo del usuario.

¿Cómo se evidencia esta jerarquía en el código?

Los archivos del proyecto evidencian claramente la utilización activa de estas estructuras:

- `Gota.java` declara: `public abstract class Gota`.
- `GotaBuena.java`, `GotaMala.java` y `ObjetoEspecial.java` extienden `Gota` e implementan las operaciones primitivas, aportando cada una su propia cinemática, margen de colisión y efecto al interactuar con el jugador.
- `ControlMovimiento.java` define la interfaz. `ControlTeclado.java` y `ControlTactil.java` la implementan.
- En `GameLluvia.java`, se inyecta la estrategia de control correspondiente según la plataforma o el modo de juego, permitiendo su alternancia dinámica en caliente.

¿Qué principios SOLID se respetan aquí?

- **Abierto/Cerrado (OCP):** La clase `Personaje` queda cerrada a modificaciones para añadir nuevos esquemas de control (por ejemplo, un mando USB); basta con crear una nueva clase que implemente `ControlMovimiento`.
- **Responsabilidad Única (SRP):** `Gota` gestiona el algoritmo y ciclo de vida; cada subclase define su cinemática y efecto particular; los controles solo interpretan hardware periférico.
- **Sustitución de Liskov (LSP):** Cualquier subclase de `Gota` puede sustituir a la clase base en la lista polimórfica de `Lluvia` sin quebrar la simulación.
- **Inversión de Dependencias (DIP):** `Personaje` depende de la abstracción `ControlMovimiento`, no de implementaciones concretas de teclado o librerías nativas.

¿Por qué este diseño es mejor que la alternativa con condicionales?

Si no se usara una clase abstracta ni interfaces, el código de `Lluvia` y `Personaje` tendría que preguntar el tipo de cada elemento con múltiples `instanceof` para decidir qué efecto aplicar. Eso generaría un alto acoplamiento y haría imposible añadir nuevas variedades sin modificar decenas de líneas. El diseño actual es limpio, extensible y fácil de probar.

5. GM-5: ¿Cómo se protegen los datos y se organiza el código?

¿Qué significa encapsulamiento en este proyecto?

El encapsulamiento es uno de los pilares de la orientación a objetos. En el juego, todas las clases del modelo declaran sus atributos como `private` o `protected`, ofreciendo métodos públicos únicamente cuando es estrictamente necesario bajo rigurosas reglas de negocio. Esto evita que clases externas modifiquen el estado interno de forma arbitraria o inconsistente.

Por ejemplo, la clase `Personaje` posee atributos vitales como `vidas`, `puntos`, `agua`, `sinAgua` e `invulnerabilidad` que son completamente privados. Su modificación solo es posible a través de métodos de negocio controlados:

- `recibirDanio()`: Comprueba que el personaje no esté en periodo de invulnerabilidad (`invulnerabilidad == 0`). Si es impactado, reduce exactamente una vida y activa una protección temporal de 1.2 segundos (`invulnerabilidad = 1.2f`).
- `quitarPuntos(int cantidad)`: Asegura mediante `Math.max(0, puntos - cantidad)` que el puntaje jamás adopte valores negativos.
- `colocar(float x)`: Emplea `MathUtils.clamp(x, 0, anchoMundo - ANCHO)`, garantizando matemáticamente que el personaje nunca pueda desplazarse fuera del área visible de la pantalla.
- **No exposición de referencias mutables:** `Personaje` no entrega su objeto geométrico interno `Rectangle area`. Para las colisiones expone el método booleano `choca(Rectangle otra)`, resguardando su integridad física. Asimismo, en `Lluvia.java`, la colección de gotas se expone a la vista mediante `Collections.unmodifiableList(gotas)`, imposibilitando que la vista altere la lista de entidades activas.

¿Qué principios SOLID y patrones GRASP se aplicaron?

El diseño arquitectónico adoptó los cinco principios SOLID y las directrices GRASP formuladas por Craig Larman:

1. **Experto en Información (GRASP):** Cada clase custodia sus propios datos. `Personaje` es el experto que calcula su propia velocidad (`getVelocidad()`), combinando la tasa base con penalizaciones por sed y estados alterados (hielo, café, veneno). `JefeOvni` conoce su umbral de furia al 50 % de salud. `PerfilJugador` administra las tablas de récords.
2. **Alta Cohesión y Bajo Acoplamiento (GRASP):** `Lluvia` se ocupa exclusivamente de la vida y muerte de las gotas. `VistaJuego` se dedica solo al dibujado geométrico

en pantalla. `Partida` gestiona los niveles y el avance sin conocer librerías de sonido ni de texturas.

3. **Responsabilidad Única (SRP):** Cada clase posee una única razón de cambio bien acotada. La lógica de daño no depende de OpenGL ni del subsistema de audio.
4. **Abierto/Cerrado (OCP):** Incorporar un nuevo control o un nuevo tipo de objeto especial no requiere reescribir la lógica de `Personaje` ni la máquina de físicas de `Lluvia`.
5. **Sustitución de Liskov (LSP):** Todas las subclases de `Gota` (`GotaBuena`, `GotaMala`, `ObjetoEspecial`) son sustituibles en el método de actualización sin alterar la consistencia del bucle de juego.
6. **Segregación de Interfaces (ISP):** La interfaz `ControlMovimiento` es limpia y mínima: exige un único método (`actualizar()`), evitando imponer métodos innecesarios a los clientes.
7. **Inversión de Dependencias (DIP):** El modelo de dominio de alto nivel (`Personaje`) depende de la abstracción de control (`ControlMovimiento`) y no de implementaciones concretas de hardware.

¿Dónde se evidencian estas buenas prácticas en el código?

- En `Personaje.java`, todos los atributos son privados y su modificación se realiza únicamente mediante métodos de negocio como `recibirDanio()` o `quitarPuntos()`.
- En `ConfiguracionJuego.java`, el constructor es privado y el acceso se restringe a través del método estático `getInstancia()`.
- En `Lluvia.java`, se retorna una vista inmutable de la colección de entidades para proteger la integridad de las gotas activas.

¿Qué beneficios concretos aporta esta organización?

- **Extensible:** Añadir nuevos personajes a la tienda simplemente requiere agregar una entrada al enumerador `Aspecto.java` sin modificar la estructura del jugador.
- **Mantenible:** Un fallo o ajuste en la velocidad por deshidratación se corrige puntualmente en `Personaje.getVelocidad()` sin riesgo de generar efectos colaterales en la detección de colisiones.
- **Verificable mediante pruebas unitarias:** La lógica no posee acoplamientos ocultos ni variables globales no controladas, lo que permitió validar el 100 % de las 36 pruebas automatizadas del proyecto sin errores.

6. GM-6: ¿Por qué centralizar la configuración y el audio? (Singleton)

¿Qué problemas presentaba el código base original?

En el prototipo inicial de LibGDX, las variables de configuración global (como el estado de silenciamiento del audio y las mejores puntuaciones) debían ser pasadas manualmente como referencias a través de múltiples clases o se leían de forma descontrolada desde el disco.

Cada vez que una pantalla o estado del juego necesitaba silenciar la música o consultar el récord, requería recibir una copia del objeto o invocar llamadas de I/O dispersas sobre el disco. Esto generaba un alto acoplamiento, duplicación de instancias de preferencias y el riesgo constante de desincronizar el audio: una pantalla podía silenciar los efectos mientras otra mantenía una referencia antigua que reanudaba la música de fondo en un canal paralelo, generando molestas superposiciones de audio y fugas de memoria.

¿Cómo resuelve el patrón Singleton estos problemas?

El patrón Singleton garantiza que una clase posea una única instancia en toda la máquina virtual de Java y proporciona un punto de acceso global y controlado hacia ella. En el proyecto se implementó en la clase `ConfiguracionJuego`:

1. **Constructor Privado:** Impide la creación de instancias mediante el operador `new` desde cualquier otra clase del proyecto.
2. **Instancia Estática Única:** Se mantiene una variable estática privada (`private static ConfiguracionJuego instancia;`) que almacena la única referencia existente.
3. **Punto de Acceso Global con Inicialización Perezosa (*Lazy*):** El método público estático `getInstancia()` inicializa la clase la primera vez que es solicitado y entrega dicha referencia compartida a cualquier cliente.
4. **Encapsulación Atómica de Almacenamiento Local:** Administra de forma centralizada el archivo de preferencias mediante `Preferences` de LibGDX y delega la gestión de aspectos y monedas en `PerfilJugador`.

Ventajas y utilidad directa para el proyecto

- **Sincronización Total del Audio:** Al existir un único gestor de estado para el sonido, alternar el audio con la tecla `M` en cualquier pantalla (Menú, Tienda, Partida o Pausa) sincroniza de inmediato todas las pistas y efectos sin inconsistencias.
- **Eliminación del Paso de Parámetros por Constructor:** Previene tener que propagar la instancia de configuración a lo largo de diez pantallas y controladores diferentes, manteniendo las firmas de los constructores limpias y cohesivas.

- **Persistencia Atómica sin Condiciones de Carrera:** Las operaciones de guardado de monedas y récords se realizan a través de un único canal, impidiendo que dos escenas sobrescriban simultáneamente el archivo de guardado en disco.

¿Dónde se evidencia su implementación en el código?

- En `ConfiguracionJuego.java`, el constructor es privado y la instancia única se obtiene mediante el método estático `getInstancia()`. Centraliza el estado del sonido, el récord y el perfil del jugador.
- En `GameLluvia.java`, durante la inicialización en `create()`, se accede a la configuración invocando `ConfiguracionJuego.getInstancia()`, asegurando una fuente única de verdad para el audio y las preferencias.
- Al presionar la tecla de audio o registrar un nuevo récord, cualquier parte del juego interactúa directamente con los métodos de negocio de la instancia única, reflejando el cambio de inmediato en todo el sistema.

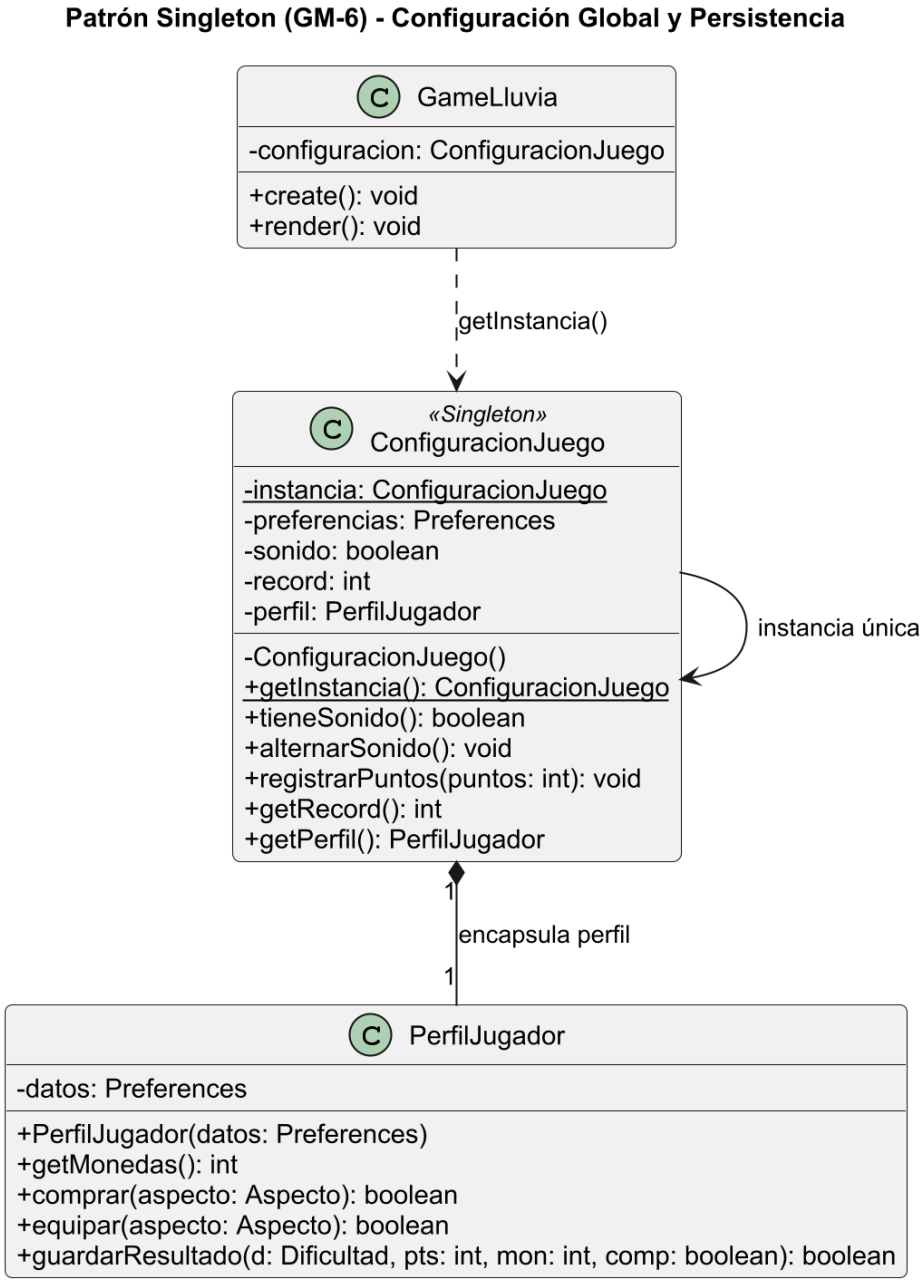


Figura 6.1: Diagrama UML del patrón Singleton aplicado a la configuración global y audio

7. GM-7: ¿Cómo se desacoplan los controles y comportamientos? (Strategy)

¿Qué limitación tenían los controles en el código base?

En el prototipo original, el tarro leía directamente las teclas de flecha (`Gdx.input.isKeyPressed(Input.Keys.LEFT)`) dentro de su propio método de actualización. Esta ligadura rígida impedía adaptar el juego a dispositivos móviles con pantalla táctil, imposibilitaba crear esquemas alternativos (como las teclas `A` y `D`) e impedía rotundamente implementar un modo multijugador local donde dos jugadores debieran compartir el mismo teclado físico con asignaciones de teclas independientes.

Si se hubiera querido añadir soporte táctil o Versus sin patrones, habría sido necesario llenar la clase del personaje con múltiples banderas booleanas y bloques condicionales dependientes de la plataforma.

¿Cómo resuelve el patrón Strategy esta situación?

El patrón Strategy define una familia de algoritmos, los encapsula en clases separadas y los hace intercambiables dinámicamente en tiempo de ejecución. En el proyecto se implementó en tres niveles complementarios de comportamiento dinámico:

1. **Estrategia de Control de Entrada (`ControlMovimiento`):**
   - **Interfaz:** Declara el método común `actualizar(Personaje personaje, float delta)`.
   - **Implementaciones concretas:** `ControlTeclado` (gestiona pulsaciones físicas para 1 o 2 jugadores) y `ControlTactil` (proyecta coordenadas táctiles en el `Viewport`).
   - **Alternancia Dinámica en Caliente:** En cualquier instante de la partida, el jugador puede pulsar la tecla `T` o tocar el botón del HUD para invocar `personaje.setControlMovimiento(crearControl())`, cambiando de inmediato el algoritmo de control sin reiniciar la partida.
2. **Estrategia de Física y Terreno (`MovimientoTerreno`):**
   - El desplazamiento del personaje adapta su algoritmo físico en tiempo de ejecución según el bioma activo mediante el método polimórfico `calcular(pedido, anterior, delta)`:
     - `NORMAL`: Traslación directa sin deformaciones.
     - `HIELO`: Incorpora inercia y deslizamiento mediante interpolación lineal (`MathUtils.lerp`), simulando patinaje sobre hielo.
     - `ATACAMA`: Aplica una componente continua de viento lateral que empuja al jugador hacia el margen derecho.
     - `ESPACIO`: Incrementa la respuesta cinemática un 20 % simulando microgravedad en órbita.

- Al avanzar de nivel y cambiar de zona climática, `Partida` inyecta en caliente la nueva estrategia en el personaje (`setMovimientoTerreno()`).

3. **Estrategia de Fases del Jefe OVNI (`JefeOvni.Fase`):**
   - La nave nodriza alienígena muta sus algoritmos de trayectoria y ataque en caliente a medida que sufre daño:
     - `NORMAL` ($> 65\,\%$ vida): Trayectoria armónica simple y cadencia regular.
     - `FURIOSA` ($\leq 65\,\%$ vida): Trayectoria compuesta caótica con oscilaciones sinusoidales superpuestas y ráfagas aceleradas orientadas a la posición del jugador.
     - `SUBORDINADOS` ($\leq 35\,\%$ vida): Despliega dos drones con escudo invulnerable y regenera vida por segundo hasta ser derribados.

¿Qué principios SOLID se benefician con este patrón?

- **Abierto/Cerrado (OCP):** La clase `Personaje` queda cerrada a modificaciones para dar soporte a nuevos dispositivos (ej. mando USB) o nuevos terrenos; basta con crear una nueva estrategia sin alterar las clases de dominio.
- **Responsabilidad Única (SRP):** La cinemática del personaje se aísla de la lectura de hardware de LibGDX y de la física específica de los escenarios.
- **Inversión de Dependencias (DIP):** El personaje depende exclusivamente de abstracciones (`ControlMovimiento` y `MovimientoTerreno`), no de controladores concretos.

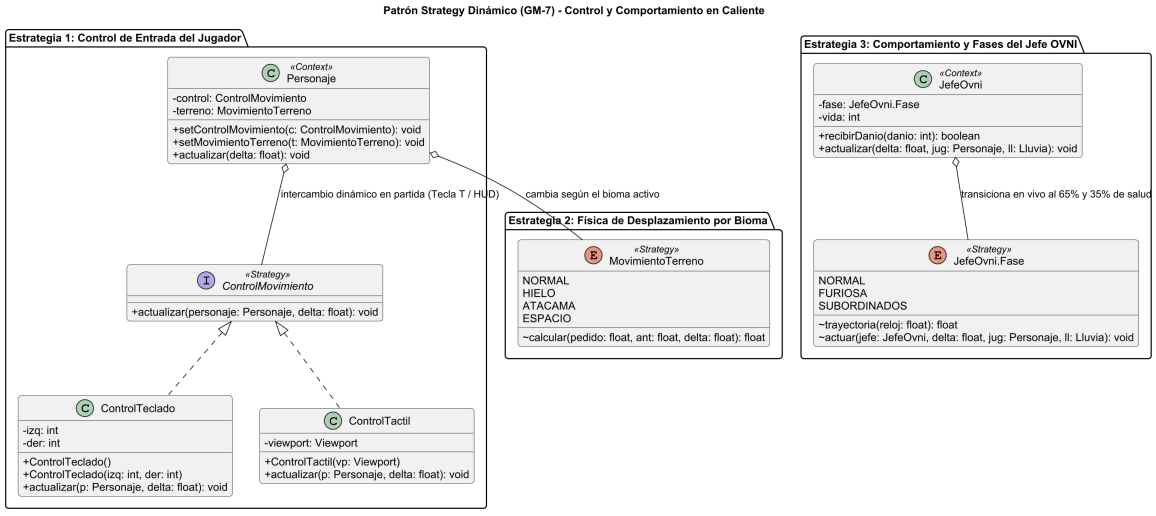


Figura 7.1: Diagrama UML del patrón Strategy aplicado a los controles dinámicos, física de terrenos y fases del jefe

8. GM-8: ¿Cómo se define el esqueleto de la caída y captura? (Template Method)

¿Qué problema presentaba la lógica de colisión en el código base?

En el prototipo original, el movimiento y la colisión de las gotas se gestionaban dentro de un extenso bloque condicional en el bucle principal de `Lluvia`. Cada tipo de gota requería una rama `if-else` que consultaba si la gota impactaba el tarro, aplicaba manualmente el efecto (sumar puntos o restar vidas) y luego la eliminaba de la colección.

Esta aproximación provocaba una severa duplicación de código geométrico, acoplaba fuertemente la clase gestora `Lluvia` a los tipos específicos de gotas y violaba el principio de Abierto/Cerrado: añadir un nuevo elemento (como agua doble, hielo o fuego) obligaba a modificar y recompilar el bucle central del juego.

Diseño óptimo del algoritmo multietapa con operaciones primitivas y ganchos

El patrón Template Method define los pasos invariantes de un algoritmo en una operación de la clase base, postergando la definición de las etapas variables a las subclases mediante métodos abstractos y ganchos (*hooks*).

Para maximizar la fuerza conceptual del patrón y permitir que las clases hijas personalicen múltiples fases del ciclo de vida, la clase abstracta `Gota` implementa el método plantilla `actualizar()` con el modificador `final`, articulado en cuatro operaciones primitivas bien diferenciadas:

1. **Paso 1: Cálculo Cinemático (`calcularMovimiento(delta)`):** *Operación primitiva abstracta.* Cada subclase personaliza su física de caída. Las gotas normales aplican descenso lineal estándar, mientras que variantes especiales pueden modificar su aceleración o aplicar factores de velocidad mediante el método protegido `caer(delta, factor)`.
2. **Paso 2: Evaluación de Escape (Paso Invariante):** Comprueba si el elemento rebasó el límite inferior del mundo (`area.y + area.height <0`), marcándolo como `terminada = true` para su liberación en memoria.
3. **Paso 3: Verificación Geométrica de Contacto (`verificarColision(personaje)`):** *Operación primitiva abstracta.* Cada elemento define su propia tolerancia y precisión de impacto mediante la función utilitaria `toca(personaje, margen)`, permitiendo márgenes más amplios para el agua vital o detección estricta para proyectiles corrosivos.
4. **Paso 4: Aplicación del Efecto de Dominio (`aplicarEfecto(personaje)`):** *Operación primitiva abstracta.* Se ejecuta obligatoriamente si hubo colisión: `GotaBuena` incrementa reservas de agua y puntaje; `GotaMala` inflige daño o veneno; y

`ObjetoEspecial` transfiere potenciadores al jugador.

5. **Paso 5: Notificación Posterior (`despuesDeAtrapar(personaje)`):** *Método Gancho (Hook).* Posee una implementación por defecto vacía. Subclases como `GotaBuena` lo sobrescriben opcionalmente para registrar avisos en pantalla (*HUD feedback*) o activar multiplicadores de carga de arma.
6. **Paso 6: Descarte Seguro (Paso Invariante):** Marca `terminada = true`, garantizando que ninguna entidad pueda aplicar su efecto dos veces en el mismo fotograma.

¿Qué principios SOLID se favorecen con este diseño?

- **Responsabilidad Única (SRP):** La clase base `Gota` gobierna la secuencia algorítmica y el descarte; las clases derivadas solo se ocupan de personalizar sus operaciones primitivas.
- **Abierto/Cerrado (OCP):** Incorporar un nuevo elemento no exige tocar el algoritmo plantilla ni la clase `Lluvia`; basta con extender `Gota` e implementar las operaciones primitivas.
- **Sustitución de Liskov (LSP):** El bucle polimórfico de `Lluvia` invoca `gota.actualizar(delta, personaje)` con total transparencia para cualquier subclase.

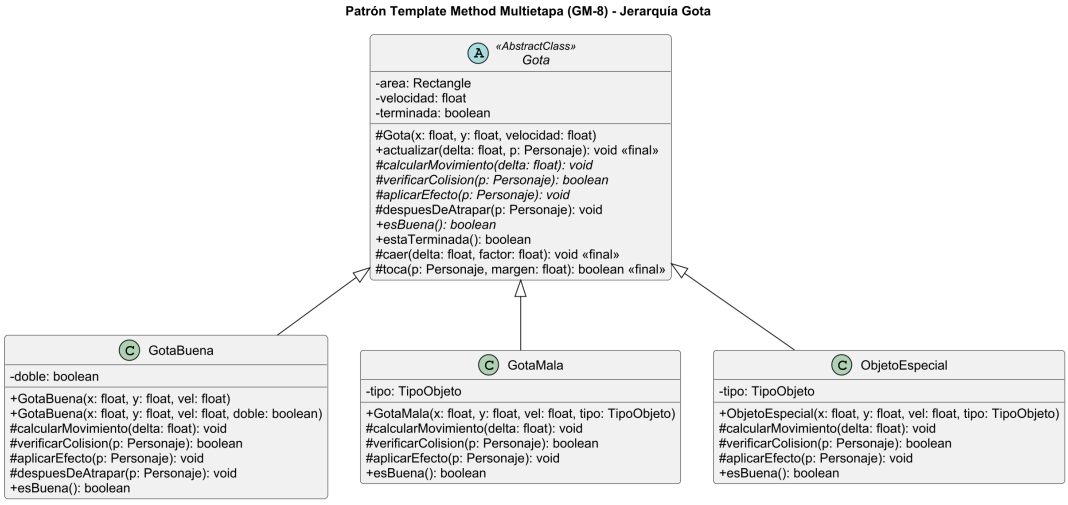


Figura 8.1: Diagrama UML del patrón Template Method multietapa en la jerarquía Gota

9. GM-9: ¿Cómo se construyen niveles complejos de forma legible? (Builder)

¿Qué dificultad presentaba la creación de niveles en el código base?

En el prototipo original, los parámetros de dificultad (como velocidades de caída, intervalos de tiempo y probabilidad de gotas malas) eran constantes numéricas fijas o estaban dispersas en varias líneas dentro de la clase `Lluvia`. Para escalar el juego y soportar una campaña de 30 niveles procedurales dividida en 3 dificultades (`NOOB`, `PRO`, `HACKER`), se requería configurar múltiples valores por cada nivel: nombre, umbral acumulado de puntos, velocidad de caída, intervalo entre gotas y probabilidad estadística de amenazas.

Una aproximación ingenua habría sido crear un constructor telescópico con múltiples parámetros contiguos del mismo tipo primitivo (`new Nivel("HACKER / 25", 2500, 580f, 0.22f, 0.42f)`). Este antipatrón resulta sumamente propenso a errores humanos involuntarios (como invertir velocidad con intervalo), reduce la legibilidad y no provee validación antes de crear el objeto.

¿Cómo resuelve el patrón Builder este problema?

El patrón Builder separa la construcción de un objeto complejo de su representación final, permitiendo crear diferentes configuraciones paso a paso mediante una interfaz fluida (*fluent interface*).

En el proyecto se diseñó la clase estática interna `Nivel.Builder`:

1. Se crea una instancia del Builder especificando el identificador del nivel (`nombre`).
2. Se invocan los métodos de configuración deseados (`objetivo()`, `velocidad()`, `intervalo()`, `probabilidadMala()`), cada uno retornando `this` para permitir encadenamiento fluido.
3. Finalmente, se invoca el método terminal `construir()`, el cual valida rigurosamente las invariantes numéricas (valores positivos finitos y probabilidades acotadas entre 0 y 1). Si los datos son válidos, instancia el objeto inmutable `Nivel`.
4. **Rol de Director en `Dificultad`:** El enum `Dificultad` actúa como *Director* formal, encapsulando las fórmulas matemáticas de progresión y configurando el Builder para generar automáticamente los 30 niveles de la campaña.

¿Dónde se evidencia esta implementación en el código?

- `Nivel.Builder`: clase estática anidada que mantiene los parámetros y ofrece métodos encadenables que retornan `this`.
- `Nivel.java`: su constructor es privado y recibe una instancia de `Builder`, garanti-

zando que ninguna clase externa pueda instanciar un nivel sin pasar por el proceso de construcción y validación.

- En `Dificultad.java`, el método `crearNivel(numero)` utiliza el builder de forma declarativa y expresiva.

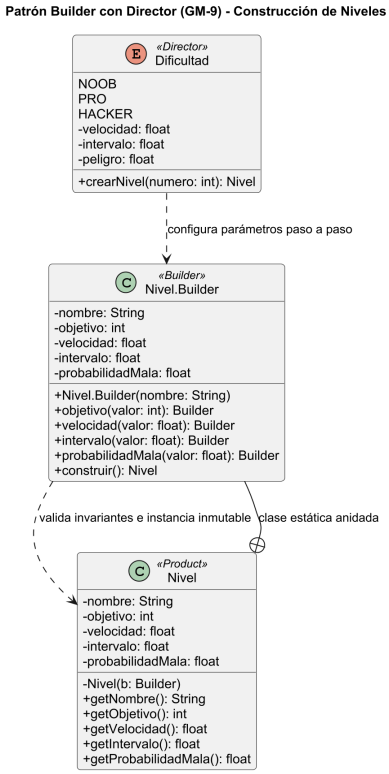


Figura 9.1: Diagrama UML del patrón Builder con Director en el enum Dificultad

10. Control de Versiones

El ciclo de desarrollo del proyecto se gestionó mediante el sistema de control de versiones distribuido Git y la plataforma remota GitHub, manteniéndose disponible en el repositorio público oficial:

`https://github.com/maverick1006/Proyecto-Game-LLuvia-`

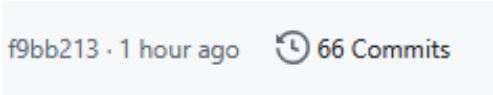


Figura 10.1: Trazabilidad e historial de commits en GitHub